

Projektdokumentation

IT-Support mit Zammad, Docker, Git, API-Integration und Knowledge Base

Projekt: Einführung eines internen Ticketsystems für die OfficeLine GmbH

System: Zammad mit Docker Compose

Bearbeiter: Muhammed Bagci

Datum: Juni 2026

1. Ausgangssituation

Die fiktive Firma OfficeLine GmbH beschäftigt 85 Mitarbeitende. Vor Projektbeginn wurden IT-Anfragen über verschiedene Wege gemeldet, zum Beispiel per E-Mail, Telefon, Teams-Nachricht oder durch persönliche Nachfrage.

Dadurch entstanden mehrere Probleme:

- IT-Anfragen waren nicht zentral dokumentiert.
- Zuständigkeiten waren unklar.
- Rückfragen gingen teilweise verloren.
- Wiederkehrende Probleme wurden mehrfach neu erklärt.
- Es gab keine einheitliche Übersicht über offene und erledigte Supportfälle.

Aus diesem Grund sollte ein internes Ticketsystem eingeführt werden. Als Lösung wurde Zammad gewählt, da es ein webbasiertes Ticketsystem mit Gruppen, Rollen, Triggern, Automatisierungen, Knowledge Base und API-Schnittstelle bietet.

2. Projektziel

Ziel des Projekts war es, ein lauffähiges internes Ticketsystem mit Zammad aufzubauen und fachlich nachvollziehbar zu dokumentieren.

Folgende Ziele wurden umgesetzt:

- Bereitstellung von Zammad mit Docker Compose
- Nutzung von Docker Volumes zur dauerhaften Datenspeicherung
- Versionierung über Git und GitHub
- Aufbau einer Supportstruktur mit Gruppen und Rollen
- Erstellung realistischer IT-Support-Tickets
- Einrichtung von Triggern zur automatischen Zuordnung
- Einrichtung von Automatisierungen zur Unterstützung des Ticketprozesses
- Erstellung einer Knowledge Base für Mitarbeitende
- Umsetzung einer einfachen API-Integration zur automatischen Ticketerstellung
- Dokumentation von Fehlern, Tests, Backup und Betrieb

- Vorbereitung einer Präsentation mit Live-Demo

3. Projektumgebung

Das Projekt wurde in einer lokalen Linux-Umgebung unter WSL umgesetzt.

Verwendete Werkzeuge:

Werkzeug	Zweck
Docker	Ausführen der Container
Docker Compose	Starten mehrerer Container als gemeinsame Umgebung
Git	Versionierung des Projekts
GitHub	Ablage des Projekt-Repositories
Zammad	Ticketsystem
Bash/curl	API-Test und automatische Ticketerstellung
VS Code	Bearbeitung von Skripten und Dokumentation

Zu Beginn wurde geprüft, ob Docker, Docker Compose und Git vorhanden sind.

Verwendete Befehle:

```
hacimah@TABLET-I2N3ACEC:~/Abschlussprojekt_IT-Support/docker$ docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu3~24.04.2
hacimah@TABLET-I2N3ACEC:~/Abschlussprojekt_IT-Support/docker$ docker-compose --version
docker-compose version 1.29.2, build unknown
hacimah@TABLET-I2N3ACEC:~/Abschlussprojekt_IT-Support/docker$ git --version
git version 2.43.0
```

Ergebnis:

- Docker war installiert.
- Docker Compose war installiert, musste aber mit docker-compose statt docker compose aufgerufen werden.
- Git war installiert.

4. Projektstruktur

Für das Projekt wurde ein eigener Ordner erstellt:

~/Abschlussprojekt_IT-Support

Die Projektstruktur wurde wie vorgegeben folgt aufgebaut:

Abschlussprojekt_IT-Support

├─ backup-tests

├─ docker

```
| └─ docker-compose.yml
|
| └─ docs
|
|   └─ fehler
|
|   └─ prozesse
|
|   └─ tests
|
|   └─ knowledge-base
|
|   └─ presentation
|
|   └─ screenshots
|
|   └─ scripts
|
|     └─ Ticket_erstellen.sh
|
|     └─ .env
|
|     └─ .gitignore
|
|     └─ README.md
```

Die Trennung der Ordner ist wichtig, weil technische Dateien, Dokumentation, Skripte, Screenshots und Präsentationsunterlagen dadurch sauber voneinander getrennt sind. Dadurch bleibt das Projekt übersichtlich und nachvollziehbar.

5. Git und GitHub

Zur Versionierung wurde Git verwendet. Das Projekt wurde zusätzlich mit einem GitHub-Repository verbunden.

Verwendete Befehle:

```
git init
```

```
git branch -M main
```

```
git add .
```

```
git commit -m "Projektstruktur und Zammad Docker Compose eingerichtet"
```

```
git remote add origin https://github.com/MhmmdBagci/Abschlussprojekt-IT-Support.git
```

```
git push -u origin main
```

Beim ersten Push trat ein Konflikt auf, weil das GitHub-Repository bereits einen ersten Commit enthielt. Die lokale und entfernte Historie mussten zusammengeführt werden.

Lösung:

```
git pull origin main --allow-unrelated-histories --no-rebase
```

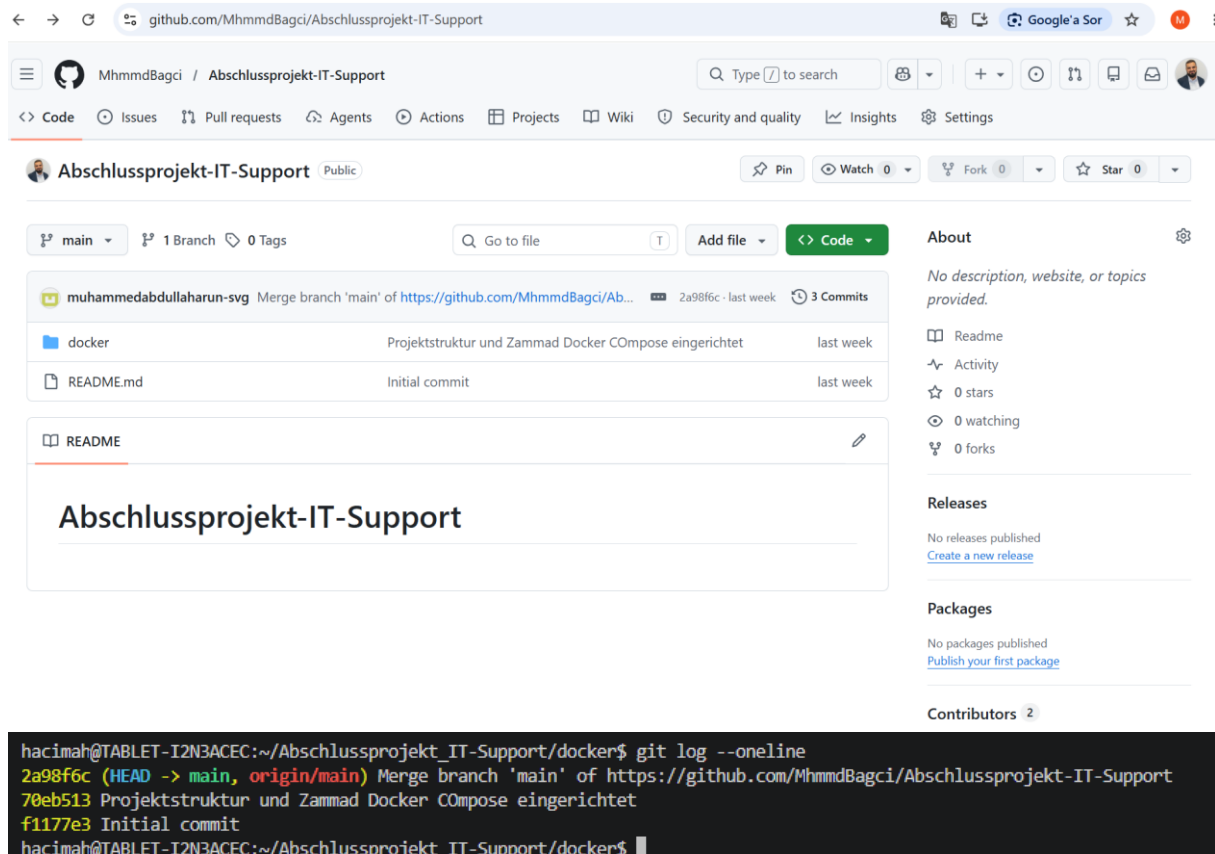
```
git push -u origin main
```

Danach war das lokale Projekt erfolgreich mit GitHub verbunden.

Wichtige Sicherheitsmaßnahme:

Die Datei `scripts/.env` wurde in `.gitignore` aufgenommen, damit Passwörter und Zugangsdaten nicht in Git gespeichert werden.

`scripts/.env`



github.com/MhmmmdBagci/Abschlussprojekt-IT-Support

MhmmmdBagci / Abschlussprojekt-IT-Support

<> Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

Abschlussprojekt-IT-Support Public

main 1 Branch 0 Tags

Go to file Add file <> Code

muhammedabdullaharun-svg Merge branch 'main' of https://github.com/MhmmmdBagci/Ab... 2a98f6c - last week 3 Commits

docker Projektstruktur und Zammad Docker Compose eingerichtet last week

README.md Initial commit last week

README

Abschlussprojekt-IT-Support

About

No description, website, or topics provided.

Readme Activity 0 stars 0 watching 0 forks

Releases

No releases published Create a new release

Packages

No packages published Publish your first package

Contributors 2

```
hacimah@TABLET-I2N3ACEC:~/Abschlussprojekt_IT-Support/docker$ git log --oneline
2a98f6c (HEAD -> main, origin/main) Merge branch 'main' of https://github.com/MhmmmdBagci/Abschlussprojekt-IT-Support
70eb513 Projektstruktur und Zammad Docker Compose eingerichtet
f1177e3 Initial commit
hacimah@TABLET-I2N3ACEC:~/Abschlussprojekt_IT-Support/docker$
```

6. Installation von Zammad mit Docker Compose

Aus der offiziellen Zammad Docker-Compose Dokumentation und dem offiziellen GitHub-Repository von Zammad <https://github.com/zammad/zammad-docker-compose/blob/master/docker-compose.yml>. Ich habe die Vorlage übernommen, anschließend mit Docker Compose gestartet." Die Docker-Compose-Datei wurde im Ordner `docker` abgelegt. Anschließend wurde die Umgebung gestartet.

Verwendete Befehle:

```
docker-compose up -d
```

```
docker-compose ps
```

Beim ersten Start wurden mehrere Container heruntergeladen und gestartet.

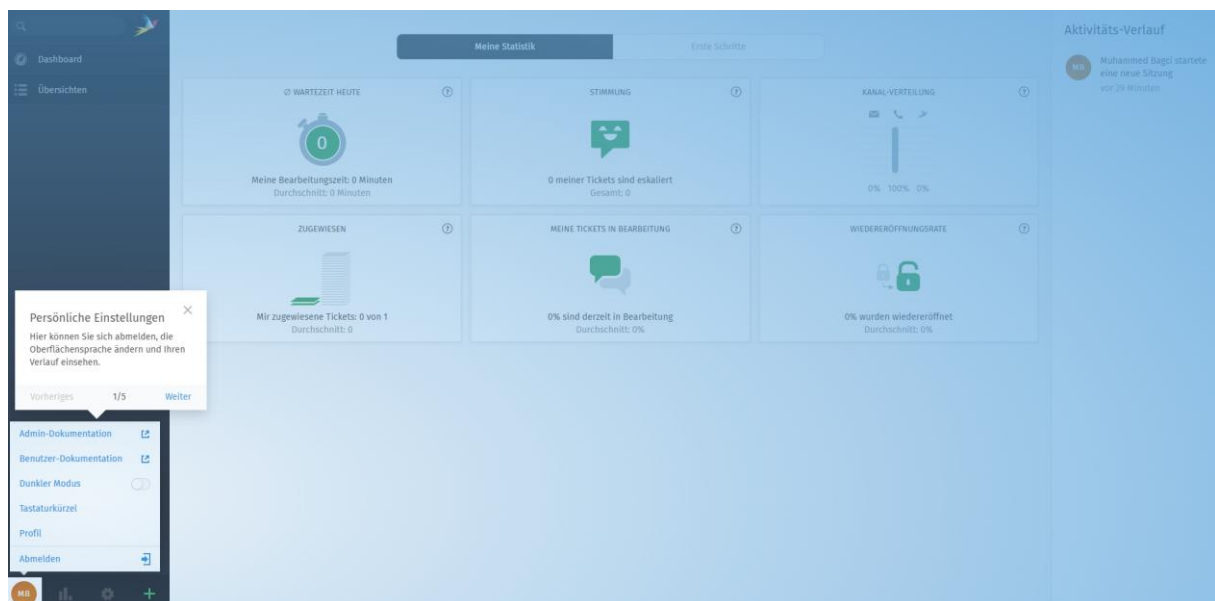
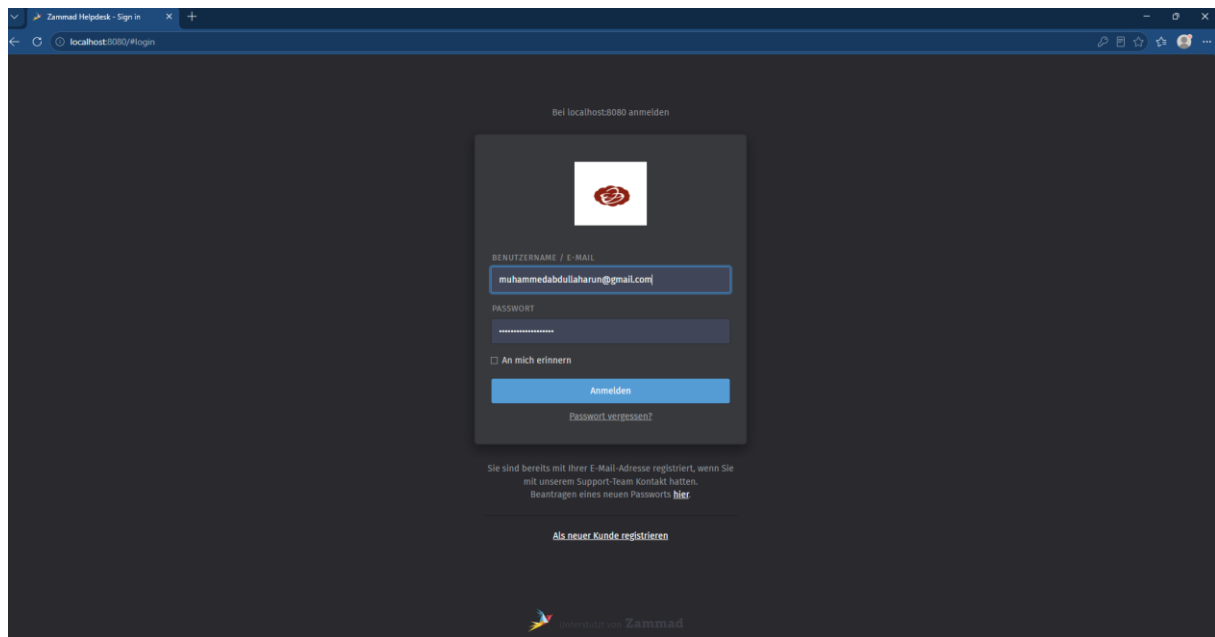
```

hacimah@TABLET-12N3ACEC:~/Abschlussprojekt_IT-Support/docker$ docker-compose ps
-----
Name                                Command                                State      Ports
-----
docker_zammad-backup_1              /opt/zammad/bin/docker-ent ...      Up
docker_zammad-elasticsearch_1       /bin/tini -- /usr/local/bi ...      Up
docker_zammad-init_1                /opt/zammad/bin/docker-ent ...      Exit 0
docker_zammad-memcached_1           docker-entrypoint.sh memca ...      Up (healthy) 11211/tcp
docker_zammad-nginx_1               /opt/zammad/bin/docker-ent ...      Up
docker_zammad-postgresql_1          docker-entrypoint.sh postgre ...      Up (healthy) 0.0.0.0:8080->8080/tcp,:::8080->8080/tcp
docker_zammad-railsserver_1         /opt/zammad/bin/docker-ent ...      Up (healthy) 5432/tcp
docker_zammad-redis_1               docker-entrypoint.sh redis ...      Up (healthy) 6379/tcp
docker_zammad-scheduler_1           /opt/zammad/bin/docker-ent ...      Up
docker_zammad-websocket_1           /opt/zammad/bin/docker-ent ...      Up
hacimah@TABLET-12N3ACEC:~/Abschlussprojekt_IT-Support/docker$ docker ps
CONTAINER ID   IMAGE                                COMMAND                                CREATED      STATUS      PORTS                                NAMES
a414bef92452   ghcr.io/zammad/zammad:7.0.1-0053   "/opt/zammad/bin/doc..."            3 days ago   Up 3 days   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp   docker_zammad-nginx_1
e2f4ca73fd85   ghcr.io/zammad/zammad:7.0.1-0053   "/opt/zammad/bin/doc..."            3 days ago   Up 3 days                                docker_zammad-backup_1
9f1764d81588   ghcr.io/zammad/zammad:7.0.1-0053   "/opt/zammad/bin/doc..."            3 days ago   Up 3 days                                docker_zammad-scheduler_1
9f576f8194bc   ghcr.io/zammad/zammad:7.0.1-0053   "/opt/zammad/bin/doc..."            3 days ago   Up 25 hours (healthy)                   docker_zammad-railsserver_1
47c30ebc3aab   ghcr.io/zammad/zammad:7.0.1-0053   "/opt/zammad/bin/doc..."            3 days ago   Up 25 hours                             docker_zammad-websocket_1
fd9d3031b135   memcached:1.6.42-alpine            "docker-entrypoint.s..."            3 days ago   Up 3 days (healthy)                    11211/tcp   docker_zammad-memcached_1
c282383c9443   postgres:17.10-alpine              "docker-entrypoint.s..."            3 days ago   Up 3 days (healthy)                    5432/tcp   docker_zammad-postgresql_1
9f887e73c6d6   elasticsearch:9.4.2                "/bin/tini -- /usr/L..."            3 days ago   Up 3 days                                docker_zammad-elasticsearch_1
50e60eb45500   redis:8.8.0-alpine                 "docker-entrypoint.s..."            3 days ago   Up 3 days (healthy)                    6379/tcp   docker_zammad-redis_1
hacimah@TABLET-12N3ACEC:~/Abschlussprojekt_IT-Support/docker$

```

Zammad ist unter folgendem Port erreichbar:

<http://localhost:8080>



7. Bedeutung der Datenbank und des Suchdienstes

Zammad benötigt eine Datenbank, weil dort alle wichtigen Informationen gespeichert werden. Dazu gehören Tickets, Benutzer, Rollen, Gruppen, Kundenantworten, interne Notizen, Einstellungen und Knowledge-Base-Inhalte.

Ohne Datenbank wären die Supportdaten nicht dauerhaft verfügbar.

Elasticsearch wird als Suchdienst verwendet. Dadurch können Tickets, Inhalte und andere Daten schneller durchsucht werden. Gerade bei vielen Tickets ist eine schnelle Suche wichtig, damit Supportmitarbeiter frühere Fälle schnell finden können.

8. Persistenz mit Docker Volumes

Damit Daten bei einem Neustart nicht verloren gehen, werden Docker Volumes verwendet. Container können gelöscht und neu erstellt werden, die Daten bleiben aber in den Volumes erhalten.

Test der Persistenz:

1. Testticket in Zammad erstellt
2. Docker-Stack gestoppt
3. Docker-Stack neu gestartet
4. geprüft, ob das Ticket weiterhin vorhanden ist

Verwendete Befehle:

`docker-compose down`

`docker-compose up -d`

`docker-compose ps`

```

hacimah@TABLET-I2N3ACEC:~/Abschlussprojekt_IT-Support/docker$ docker-compose down
Stopping docker_zammad-nginx_1      ... done
Stopping docker_zammad-backup_1     ... done
Stopping docker_zammad-scheduler_1  ... done
Stopping docker_zammad-railsserver_1 ... done
Stopping docker_zammad-websocket_1  ... done
Stopping docker_zammad-memcached_1  ... done
Stopping docker_zammad-postgresql_1 ... done
Stopping docker_zammad-elasticsearch_1 ... done
Stopping docker_zammad-redis_1      ... done
Removing docker_zammad-nginx_1      ... done
Removing docker_zammad-init_1       ... done
Removing docker_zammad-backup_1     ... done
Removing docker_zammad-scheduler_1  ... done
Removing docker_zammad-railsserver_1 ... done
Removing docker_zammad-websocket_1  ... done
Removing docker_zammad-memcached_1  ... done
Removing docker_zammad-postgresql_1 ... done
Removing docker_zammad-elasticsearch_1 ... done
Removing docker_zammad-redis_1      ... done
Removing network docker_default
hacimah@TABLET-I2N3ACEC:~/Abschlussprojekt_IT-Support/docker$ docker-compose up -d
Creating network "docker_default" with the default driver
Creating docker_zammad-postgresql_1 ... done
Creating docker_zammad-memcached_1  ... done
Creating docker_zammad-elasticsearch_1 ... done
Creating docker_zammad-redis_1      ... done
Creating docker_zammad-scheduler_1  ... done
Creating docker_zammad-railsserver_1 ... done
Creating docker_zammad-init_1       ... done
Creating docker_zammad-backup_1     ... done
Creating docker_zammad-websocket_1  ... done

ERROR: for zammad-nginx Container "5e479e3da82f" is unhealthy.
ERROR: Encountered errors while bringing up the project.

```

Ergebnis:

Das Testticket war nach dem Neustart weiterhin vorhanden. Damit wurde nachgewiesen, dass die Daten dauerhaft gespeichert werden. PS: NGINX hat jedes Mal nicht auf antrieb gestartet ich musste immer etwas warten und ihn dann Manuel nochmal starten. Die Website wird dann auch nicht direkt erreicht auch wenn es dann läuft immer nach cookies

löschen hat es dann aber wieder funktioniert.

```
nacimah@TABLET-I2N3ACEC:~/Abschlussprojekt_IT-Support/docker$ docker-compose up -d zammad-nginx
docker_zammad-postgresql_1 is up-to-date
docker_zammad-memcached_1 is up-to-date
docker_zammad-redis_1 is up-to-date
docker_zammad-railsserver_1 is up-to-date
Creating docker_zammad-nginx_1 ... done
nacimah@TABLET-I2N3ACEC:~/Abschlussprojekt_IT-Support/docker$ docker-compose ps
```

Name	Command	State	Ports
docker_zammad-backup_1	/opt/zammad/bin/docker-ent ...	Up	
docker_zammad-elasticsearch_1	/bin/tini -- /usr/local/bi ...	Up	9200/tcp, 9300/tcp
docker_zammad-init_1	/opt/zammad/bin/docker-ent ...	Up	
docker_zammad-memcached_1	docker-entrypoint.sh memca ...	Up (healthy)	11211/tcp
docker_zammad-nginx_1	/opt/zammad/bin/docker-ent ...	Up	0.0.0.0:8080->8080/tcp, :::8080->8080/tcp
docker_zammad-postgresql_1	docker-entrypoint.sh postgre...	Up (healthy)	5432/tcp
docker_zammad-railsserver_1	/opt/zammad/bin/docker-ent ...	Up (healthy)	
docker_zammad-redis_1	docker-entrypoint.sh redis ...	Up (healthy)	6379/tcp
docker_zammad-scheduler_1	/opt/zammad/bin/docker-ent ...	Up	
docker_zammad-websocket_1	/opt/zammad/bin/docker-ent ...	Up	

9. Zammad-Ersteinrichtung

Nach erfolgreichem Start wurde Zammad im Browser geöffnet:

<http://localhost:8080>

Anschließend wurde das System neu eingerichtet und ein Administrator-Konto erstellt.

Admin-Benutzer:

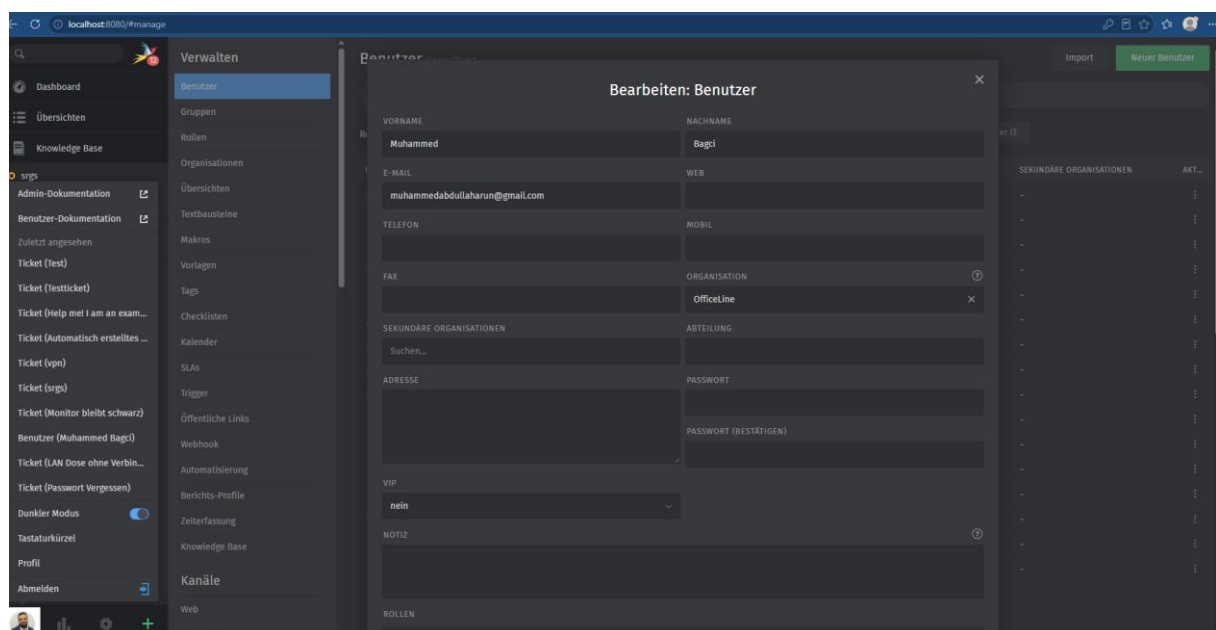
Feld Wert

Vorname Muhammed

Nachname Bagci

E-Mail muhammedabdullaharun@gmail.com

Rolle Administrator



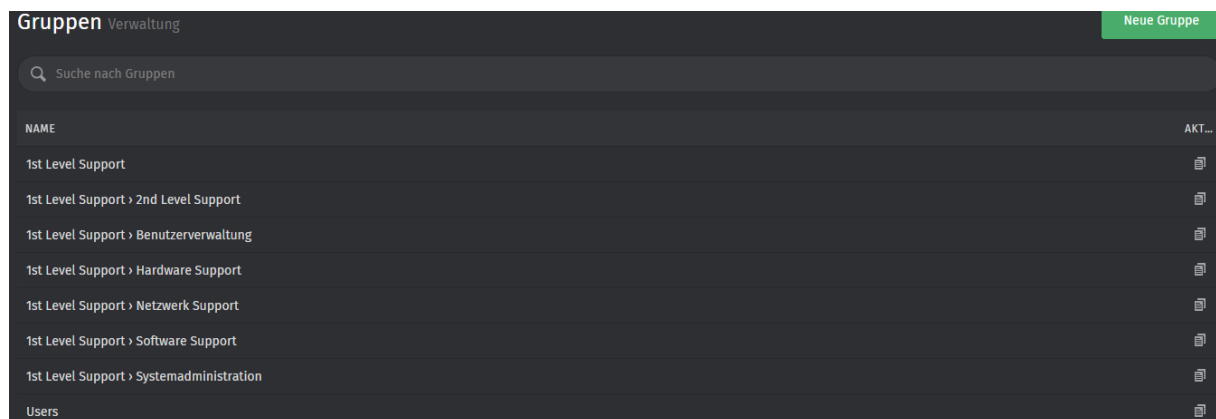
10. Supportorganisation

Für die OfficeLine GmbH wurde eine Supportstruktur erstellt. Ziel war es, Anfragen fachlich sinnvoll zu verteilen.

Angelegte Gruppen:

Gruppe	Aufgabe
1st Level Support	Erste Annahme und Vorprüfung von Tickets
Benutzerverwaltung	Passwort, Benutzerkonten, Zugriffe, Berechtigungen
Hardware Support	Drucker, Scanner, Laptop, Monitor, Headset
Software Support	Outlook, Teams, Office, Softwareinstallation
Netzwerk Support	VPN, WLAN, LAN, Internet, Gast-WLAN
2nd Level Support	Komplexe Fachanwendungen und Fehleranalyse
Systemadministration	Server, Backup, Sicherheit, kritische Störungen

Die Gruppen wurden nicht nur angelegt, weil sie in der Aufgabe gefordert waren, sondern weil sie fachlich unterschiedliche Aufgaben im IT-Support abbilden.



11. Rollen und Rechte

Zur Trennung von Zuständigkeiten wurden Rollen definiert.

Rolle	Darf machen	Darf nicht machen	Begründung
Support Agent	Tickets bearbeiten, Kunden antworten	Systemeinstellungen ändern	Für normale Supportarbeit
Senior Support Agent	komplexe Tickets bearbeiten, eskalierte Fälle übernehmen	Grundsystem verändern	Für erfahrene Supportfälle
Knowledge Base Editor	KB-Artikel erstellen und bearbeiten	Systemadministration durchführen	Für Wissensdokumentation

Rolle	Darf machen	Darf nicht machen	Begründung
Teamleiter IT	Tickets überwachen, Gruppen steuern	technische Systembasis ändern	Für Steuerung und Kontrolle
System Administrator	System verwalten, technische Einstellungen ändern	keine Einschränkung im Projekt	Für Administration notwendig

NAME	AKTIV BEI NEUANMELDUNG	AKT...
Knowledge Base Editor	nein	
Kunde	ja	
Senior Support Agent	nein	
Support Agent	nein	
Systemadministrator	nein	
Teamleiter IT	nein	

12. Benutzer und Kunden

Es wurden interne Kunden und Supportmitarbeiter angelegt. Dadurch konnten Tickets realistisch Kunden, Abteilungen und Bearbeitern zugeordnet werden.

Beispiele:

Name	Rolle im Szenario	Gruppe/Abteilung
Max Berger	IT-Leiter	IT
Sven Müller	Systemadministrator	Systemadministration
Leon Weber	Senior Support	2nd Level Support
Nina Koch	Support Agentin	1st Level Support
Anna Keller	Kundin	Buchhaltung
Jonas Hartmann	Kunde	Vertrieb
Miriam Vogel	Kundin	Personal
Lisa Neumann	Kundin	Geschäftsführung
David Braun	Kunde	Lager
Paul Schneider	Kunde	Kundenservice

Verwalten

Benutzer

Gruppen

Rollen

Organisationen

Übersichten

Textbausteine

Makros

Vorlagen

Tags

Checklisten

Kalender

SLAs

Trigger

Öffentliche Links

Webhook

Automatisierung

Berichts-Profile

Zeiterfassung

Knowledge Base

Kanäle

Benutzer Verwaltung

Import

Neuer Benutzer

Benutzer suchen

Rollen: Knowledge Base-BearbeiterKundeSenior Support AgentSupport AgentSystemadministratorTeamleiter IT

LOGIN	VORNAME	NACHNAME	ORGANISATION	SEKUNDÄRE ORGANISATIONEN	AKT...
api.integration@officeline.local	API	Integration	-	-	⋮
lisa.neumann@officeline.de	Lisa	Neumann	OfficeLine	-	⋮
paul.schneider@officeline.de	Paul	Schneider	OfficeLine	-	⋮
david.braun@officeline.de	David	Braun	OfficeLine	-	⋮
miriam.vogel@officeline.de	Miriam	Vogel	OfficeLine	-	⋮
jonas.hartmann@officeline.de	Jonas	Hartmann	OfficeLine	-	⋮
anna.keller@officeline.de	Anna	Keller	OfficeLine	-	⋮
auto-b74fe1-1d2c-4182-a2e1-bd6...	Laura	Becker	OfficeLine	-	⋮
sven.müller@officeline.de	Sven	Müller	OfficeLine	-	⋮
maja.klein@officeline.de	Maja	Klein	OfficeLine	-	⋮
leon.weber@officeline.de	Leon	Weber	OfficeLine	-	⋮
nina.koch@officeline.de	Nina	Koch	OfficeLine	-	⋮
max.berger@officeline.de	Max	Berger	OfficeLine	-	⋮
test@test.de	Testkunde	Test	OfficeLine	-	⋮
muhammedabdullaharun@gmail.c...	Muhammed	Bagci	OfficeLine	-	⋮
nicole.braun@zammad.org	Nicole	Braun	Zammad Foundation	-	⋮

13. Ticketmodell

Vor der Ticketerstellung wurde ein einheitliches Ticketmodell verwendet. Dadurch sind alle Tickets vergleichbar und nachvollziehbar.

Bearbeiten: Vorlage

NAME *

Ticketmodell - OfficeLine

AKTIONEN

Text

Ticket Titel	System + Symptom + Kontext
Benutzer	
Abteilung	
Gerät (PC Name / Inventar)	
Seit wann	
Ort (Büro, Homeoffice, Standort)	

1. Fehlerbild

Fehlerbild

2. Umgebung

Umgebung Details

• Windows Version:

• Programm und Version:

• VPN: an oder aus

• Netzwerk: WLAN, LAN, VPN

3. Bereits getestet

Getestete Schritte und Ergebnisse

4. Beobachtung und Analyse

Beobachtung und Analyse

5. Anhänge

Anhänge

-

6. Nächster Schritt oder Eskalation

Nächster Schritt oder Eskalation

Titel

—

+

Kunde

Person oder Organisation/Firma eingeben

—

+

Organisation

Suchen...

—

+

Priorität

1 niedrig

—

+

Gruppe

—

+

Status

neu

—

+

AKTIV *

aktiv

Abbrechen & Zurück

Übermitteln

PS

T01 Passwort vergessen

Ticket#40050 @ - erstellt vor 8 Stunden 35 Minuten

Ticket Titel	T01 Passwort vergessen
Benutzer	Paul Schneider
Abteilung	Kundenservice
Gerät (PC Name / Inventar)	PC-KS-021
Seit wann	Seit heute Morgen 08:00 Uhr
Ort (Büro, Homeoffice, Standort)	Büro

1. Fehlerbild

Fehlerbild

Der Benutzer kann sich nicht mehr am Windows-System anmelden.
Nach mehreren Fehlversuchen wurde das Passwort vergessen.
Der Zugriff auf Outlook und interne Anwendungen ist dadurch nicht möglich.

2. Umgebung

Umgebung Details

• VPN: an
• Netzwerk: WLAN

3. Bereits getestet

Getestete Schritte und Ergebnisse

- Benutzername korrekt
- Tastaturbelegung geprüft
- Keine technische Störung vorhanden

4. Beobachtung und Analyse

Beobachtung und Analyse

Benutzer telefonisch kontaktiert. Identität geprüft. Auf den Artikel "Passwort selbst zurücksetzen" im Knowledge Base verwiesen.

5. Anhänge

Anhänge

-

6. Nächster Schritt oder Eskalation

Schreibe, was als nächstes passieren soll und an wen es geht.

Nächster Schritt oder Eskalation

Der Benutzer wurde auf den Knowledge Base Artikel "Passwort selbst zurücksetzen" verwiesen.
Der Benutzer konnte sein Passwort über das Self-Service Portal erfolgreich zurücksetzen.
Anmeldung an Windows und Outlook wieder möglich.

[Notiz anzeigen](#)

 auf intern setzen

 abspalten

vor 8 Stunden 35 Minuten

Notiz schreiben, oder Dateien wählen...

14. Tickets

Es wurden 40 realistische Tickets erstellt. Die Tickets decken typische IT-Supportfälle ab:

- Passwort vergessen
- Benutzerkonto gesperrt
- neuer Mitarbeiter
- Austritt eines Mitarbeiters
- Berechtigungen
- SharePoint-Zugriff
- VPN-Probleme
- WLAN-Probleme
- Druckerprobleme
- Hardwareprobleme
- Softwareprobleme
- Outlook/Teams-Probleme
- Phishing-Verdacht
- Server- und Netzwerkstörungen

Die Tickets wurden unterschiedlich bearbeitet:

Anforderung	Umsetzung
40 Tickets insgesamt	erstellt
Mindestens 15 vollständig gelöst	umgesetzt
Mindestens 8 eskaliert	umgesetzt
Mindestens 5 warten auf Kunde	umgesetzt
Mindestens 10 mit Knowledge-Base-Bezug	umgesetzt
Mindestens 3 kritische Tickets	umgesetzt

Screenshot einfügen:

Screenshot: Ticketübersicht mit 40 Tickets

15. Vollständig gelöste Tickets

Mehrere Tickets wurden vollständig bearbeitet und geschlossen. Dabei wurden Analyse, Lösungsschritte, Kundenantwort, interne Notiz und Ergebnis dokumentiert.

PS

ils

T01 Passwort vergessen

TicketID:0001

erstellt vor 7 Stunden

Ticket Titel	T01 Passwort vergessen
Betreiber	Paul Schneider
Abteilung	Kundenservice
Gerät (PC, Name / Inventar)	Flimschnur
Seit wann	Seit heute morgen um 06.00 Uhr
Ort (Wohn, HomeOffice, Standort)	Büro

1. Fehlerbild

Beschreibe nur, was passiert. Gehe Vermutung.

Fehlerbild

Anmeldung hat wegen falschem Passwort fehlgeschlagen. Es betrifft nur den Mitarbeiter bzw. sein Account.

2. Umgebung

Umgebung Details

Irrelevant

3. Bereits getätigt

Liste alle Schritte auf, die du bereits durchgeführt hast. Mit Ergebnis.

Getätigte Schritte und Ergebnisse

Der Mitarbeiter erinnert sich nicht mehr nach seinem Urlaub an seinen Passwort, hat zwar versucht was er sich erinnern konnte hat aber immer die Meldung Passwort fehlgeschlagen gezeigt, wurde an die Knowledge Base "Passwort vergessen" Artikel aufmerksam gemacht.

4. Beobachtung und Analyse

Schreibe Hinweise aus Tools. Optional eine vorsichtige Vermutung.

Beobachtung und Analyse

Der Mitarbeiter wird es selbstständig versuchen mit Hilfe KI Artikel.

5. Anhänge

- Screenshots

- Logs

- Dateien

Anhänge

-

6. Nächster Schritt oder Eskalation

Schreibe, was als nächstes passieren soll und an was es geht.

Nächster Schritt oder Eskalation

Warten auf Mitarbeiter.

Benutzeranfragen

auf intern setzen

vor 7 Stunden

abgelehnt

Benutzer erfolgreich auf Knowledge Base verwiesen. Problem selbstständig gelöst.

auf öffentlich setzen

abgelehnt

Durchgeführte Schritte:

16. Eskalierte Tickets

Eine Eskalation bedeutet in diesem Projekt, dass der 1st Level Support ein Problem nicht lösen kann und das Ticket an eine passende Fachgruppe weitergibt.

Meine zugewiesenen Tickets	0	Eskalierte Tickets	OPTIONEN
Nicht zugewiesene & offene Tickets	59		
Meine "Warten erreicht" Tickets	0		
Meine abonnierten Tickets	0		
Offene Tickets	60		
"Warten erreicht" Tickets	1		
Eskalierte Tickets	10		
Meine Tickets	6		
Meine Organisations-Tickets	88		

TITEL	KUNDE	GRUPPE	BESITZER	ERSTELLT AM
Testticket	Testkunde Test (OfficeLine)	1st Level Support > Systemadministration	-	V...
Austritt eines Mitarbeiters	David Braun (OfficeLine)	1st Level Support > Systemadministration	-	V...
VPN Langsam	Maja Klein (OfficeLine)	1st Level Support > Netzwerk Support	-	V...
Laptop startet nicht	Anna Keller (OfficeLine)	1st Level Support > Systemadministration	-	V...
Office Lizenz abgelaufen	Lisa Neumann (OfficeLine)	1st Level Support > Systemadministration	-	V...
ERP Zugriff funktioniert nicht	Nina Koch (OfficeLine)	1st Level Support > Systemadministration	-	V...
Mehrere Nutzer ohne Internet	Lisa Neumann (OfficeLine)	1st Level Support > Systemadministration	-	V...
Gast WLAN Zugang benötigt	Jonas Hartmann (OfficeLine)	1st Level Support > Netzwerk Support	-	V...
Daten versehentlich gelöscht	Anna Keller (OfficeLine)	1st Level Support > Systemadministration	-	V...
Software Fehler nach Update	Laura Becker (OfficeLine)	1st Level Support > Systemadministration	-	V...

17. Tickets mit Status „Warten auf Kunde“

Mindestens fünf Tickets wurden auf „Warten auf Kunde“ gesetzt. Dabei wurde dem Kunden eine Rückfrage gestellt oder auf einen Knowledge-Base-Artikel verwiesen.

Meine zugewiesenen Tickets	0	"Warten erreicht" Tickets	OPTIONEN
Nicht zugewiesene & offene Tickets	59		
Meine "Warten erreicht" Tickets	0		
Meine abonnierten Tickets	0		
Offene Tickets	60		
"Warten erreicht" Tickets	5		
Eskalierte Tickets	10		
Meine Tickets	6		
Meine Organisations-Tickets	88		

TITEL	KUNDE	GRUPPE	BESITZER	ERSTELLT AM
LAN Dose ohne Verbindung	Paul Schneider (OfficeLine)	1st Level Support	-	vor 3 Tagen 1 Stunde
Monitor bleibt schwarz	Lisa Neumann (OfficeLine)	1st Level Support	-	vor 3 Tagen
Automatisch erstelltes Ticket: Monitoring Warnung	Muhammed Bagci (OfficeLine)	1st Level Support > Systemadministration	-	vor 1 Tag 4 Stunden
T03 Neuer Mitarbeiter braucht Konto	Maja Klein (OfficeLine)	1st Level Support	-	vor 8 Stunden 28 Min...
T39 Zugriff auf Projektgruppe beantragen	Laura Becker (OfficeLine)	1st Level Support	-	vor 4 Stunden 49 Min...

18. Kritische Tickets

Mindestens drei Tickets wurden als kritisch eingestuft.

Beispiele:

Ticket	Grund für kritische Priorität
T31 Phishing Verdacht	Sicherheitsrisiko für Benutzerkonten und Daten
T32 Mehrere Nutzer ohne Internet	mehrere Mitarbeitende können nicht arbeiten
T33 Serverdienst nicht erreichbar	mehrere Abteilungen betroffen

Beispiel T32:

Auswirkung:

Mehrere Benutzer haben keinen Internetzugriff. Dadurch sind E-Mail, Cloud-Dienste und externe Anwendungen nicht nutzbar.

Eskalationsweg:

1st Level Support → Netzwerk Support → Systemadministration informiert

Screenshot einfügen:
Screenshot: kritisches Ticket

19. Trigger

Es wurden Trigger erstellt, um Tickets automatisch zuzuordnen oder mit Tags zu versehen.

Trigger Verwaltung		Beschreibung	Neuer Trigger
Suche nach Triggern			
NAME	AKT...		
Berechtigung	⋮		
Druckerproblem	⋮		
Keine klare Kategorie	⋮		
Kritischer Vorfall	⋮		
Neuer Mitarbeiter	⋮		
Passwortproblem	⋮		
Phishing Verdacht	⋮		
Software Installation	⋮		
VPN Problem	⋮		

Bewertung der Trigger:

Trigger sind sinnvoll, weil Standardfälle schneller zugeordnet werden. Es gibt aber auch Risiken. Ein Wort wie „Zugriff“ kann mehrere Ursachen haben, zum Beispiel SharePoint, ERP oder Netzlaufwerk. Deshalb ersetzen Trigger nicht die Prüfung durch Supportmitarbeiter, sondern unterstützen nur die erste Zuordnung.

20. Automatisierungen

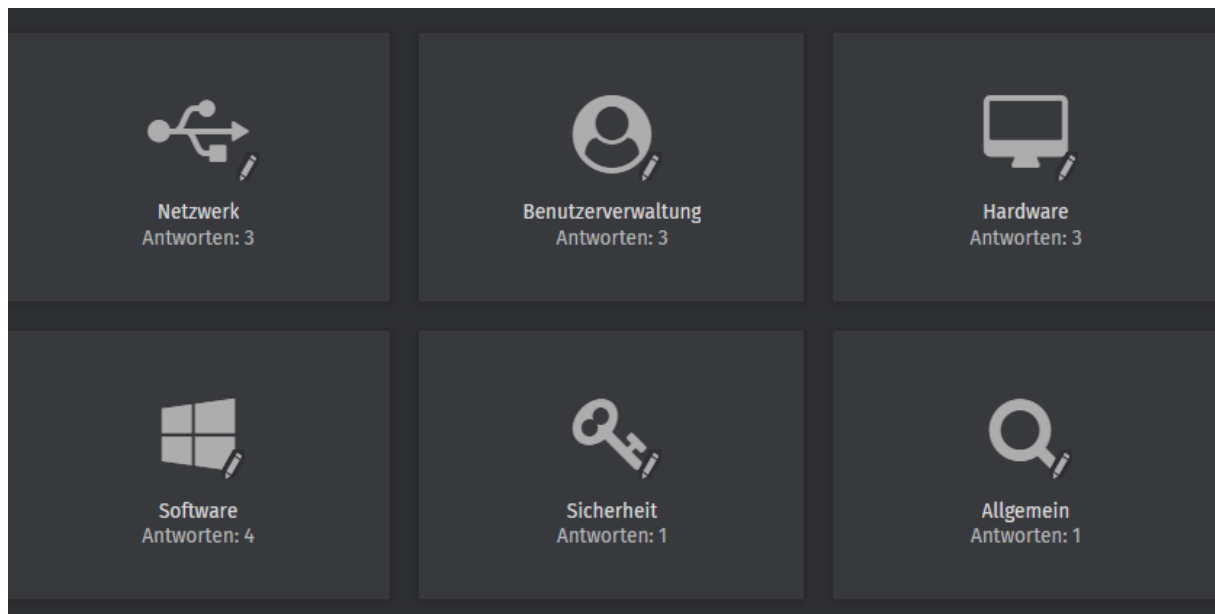
Zusätzlich wurden Automatisierungen geplant und umgesetzt.

Automatisierungen Verwaltung						Neue Automatisierung
Suche nach Automatisierungen						
NAME	OBJEKT	LETZTER LAUF	WURDE VERARBEITET	GEPLANT FÜR	WIRD VERARBEITET	AKT...
Alte offene Tickets markieren	Ticket	vor 1 Stunde 47 Minuten	28	in 39 Minuten	30	⋮
Gelöste Tickets für Knowledge Bas...	Ticket	vor 1 Stunde 42 Minuten	0	in 39 Minuten	0	⋮
Kritische Tickets täglich prüfen	Ticket	vor 1 Stunde 42 Minuten	10	in 39 Minuten	10	⋮
Onboarding Tickets priorisieren	Ticket	-	0	in 3 Tagen	0	⋮
Tickets ohne Bearbeiter finden	Ticket	vor 1 Stunde 42 Minuten	28	in 39 Minuten	30	⋮
Warten auf Kunde nachfassen	Ticket	-	0	in 3 Tagen	1	⋮

Die Automatisierung „Gelöste Tickets für Knowledge Base prüfen“ erstellt keinen Artikel automatisch. Das wäre fachlich riskant. Stattdessen markiert sie geeignete Tickets mit kb-pruefen, damit ein Knowledge-Base-Editor prüfen kann, ob ein Artikel erstellt oder angepasst werden soll.

21. Knowledge Base

Für wiederkehrende Probleme wurde eine Knowledge Base erstellt. Ziel ist es, einfache Standardfälle teilweise durch Selbsthilfe zu lösen und den IT-Support zu entlasten.



Beispielartikel:

Titel:

VPN Verbindung prüfen

Problem:

Die VPN-Verbindung kann nicht hergestellt werden.

Lösung:

- 1. Internetverbindung prüfen**
- 2. VPN-Client neu starten**
- 3. Zugangsdaten kontrollieren**
- 4. Computer neu starten**
- 5. Fehlermeldung notieren**

Wann ein Ticket erstellt werden muss:

Wenn die Verbindung weiterhin nicht möglich ist oder eine Fehlermeldung erscheint.

Die Knowledge Base wurde in mindestens zehn Tickets fachlich verwendet. Kunden wurden auf passende Artikel verwiesen und konnten einfache Probleme selbst lösen.

Screenshot einfügen:

Screenshot: Knowledge Base Übersicht

Screenshot: Beispielartikel VPN Verbindung prüfen

22. API-Integration

Für die API-Integration wurde ein Bash-Skript mit curl erstellt. Ziel war es, automatisch ein Ticket in Zammad anzulegen.

Dafür wurde der API-Zugriff in Zammad aktiviert.

Getestete Varianten:

1. HTTP Token Authentication
2. HTTP Basic Authentication

Die Token-Authentifizierung funktionierte für GET-Abfragen, zum Beispiel:

```
curl -H "Authorization: Token token=${ZAMMAD_TOKEN}"  
http://localhost:8080/api/v1/users/me
```

Bei POST-Anfragen zur Ticketerstellung traten jedoch Authentifizierungsprobleme auf. Danach wurde die Integration über HTTP Basic Authentication umgesetzt.

Die Zugangsdaten wurden nicht direkt ins Skript geschrieben, sondern in einer .env-Datei gespeichert.

Beispiel .env:

```
ZAMMAD_URL="http://localhost:8080"  
ZAMMAD_USER="muhammedabdullaharun@gmail.com"  
ZAMMAD_PASSWORD="*****"
```

Die Datei .env wurde in .gitignore eingetragen, damit Zugangsdaten nicht ins GitHub-Repository gelangen.

Beispiel Skript:

```
#!/bin/bash  
  
source "$(dirname "$0")/.env"  
  
echo "ZAMMAD API TICKET ERSTELLEN"  
echo "=====  
  
read -p "Titel des Tickets: " TITEL  
read -p "Kunde/Name: " KUNDE  
read -p "Abteilung: " ABTEILUNG  
read -p "Seit wann besteht das Problem?: " SEIT_WANN  
read -p "Betroffenes Gerät oder Anwendung: " GERAET  
read -p "Fehlermeldung oder Beobachtung: " FEHLER
```

```
read -p "Auswirkung auf die Arbeit: " AUSWIRKUNG
```

```
read -p "Was wurde bereits versucht?: " VERSUCHT
```

```
BESCHREIBUNG="Kunde: $KUNDE | Abteilung: $ABTEILUNG | Seit wann: $SEIT_WANN |  
Gerät/Anwendung: $GERAET | Fehlermeldung/Beobachtung: $FEHLER | Auswirkung:  
$AUSWIRKUNG | Bereits versucht: $VERSUCHT"
```

```
curl -X POST "$ZAMMAD_URL/api/v1/tickets" \  
-u "$ZAMMAD_USER:$ZAMMAD_PASSWORD" \  
-H "Content-Type: application/json" \  
-d "{  
  \"title\": \"$TITEL\",  
  \"customer_id\": 3,  
  \"article\": {  
    \"subject\": \"$TITEL\",  
    \"body\": \"$BESCHREIBUNG\",  
    \"type\": \"note\",  
    \"internal\": false  
  },  
  \"priority_id\": 2,  
  \"state_id\": 1  
}"
```

Was macht das Skript?

Das Skript fragt im Terminal die wichtigsten Ticketinformationen ab und erstellt anschließend über die REST-API ein neues Ticket in Zammad.

Welches Problem löst die Integration?

Externe Systeme oder einfache Formulare können automatisch Tickets erstellen, ohne dass ein Supportmitarbeiter das Ticket manuell anlegen muss.

Welche Daten werden übertragen?

- Titel
- Kunde
- Abteilung
- Zeitpunkt

- **Gerät oder Anwendung**
- **Fehlermeldung**
- **Auswirkung**
- **bereits durchgeführte Maßnahmen**

Welche Risiken gibt es?

- **Zugangsdaten dürfen nicht veröffentlicht werden.**
- **Falsche API-Daten können fehlerhafte Tickets erzeugen.**
- **Ein kompromittierter Zugang könnte missbraucht werden.**
- **Die .env-Datei darf nicht in Git gespeichert werden.**

Was passiert bei falschen Zugangsdaten?

Zammad lehnt die Anfrage ab. Es erscheint eine Fehlermeldung wie:

Invalid BasicAuth credentials

Was passiert, wenn Zammad nicht erreichbar ist?

Das Skript kann keine Verbindung herstellen. In diesem Fall gibt curl eine Verbindungsfehlermeldung aus.

Wie könnte diese Integration in einer echten Firma genutzt werden?

- **Monitoring-System erstellt ein Ticket bei Serverausfall.**
- **Backup-Skript meldet fehlgeschlagene Sicherungen.**
- **Intranet-Formular erstellt automatisch Tickets.**
- **Sicherheitssoftware meldet verdächtige Ereignisse.**

API Verwaltung

☒ Token-Zugriff (HTTP Token Authentication)

REST-API-Zugriff über Token (nicht Benutzername/E-Mail und Passwort) aktivieren. Alle Benutzer müssen persönliche Zugangs-Token im Benutzerprofil erstellen.

Beispiel:

```
curl -H "Authorization: Token token=_" http://localhost:8080/api/v1/groups
```

☐ Passwort-Zugriff (HTTP Basic Authentication)

REST-API-Zugriff mit Authentifizierung über Benutzername/E-Mail und Passwort aktivieren.

Beispiel:

```
curl -u muhammedabdullaharun@gmail.com:some_password http://localhost:8080/api/v1/groups
```

Anwendungen (OAuth Authentication)

Verwalten von Anwendungen welche Zammad als OAuth-Anbieter verwenden und Anwender autorisieren können.

KEINE EINTRÄGE

Neue Applikation

OAuth URLs sind:

AKTION	URL
Rechteanforderung	http://localhost:8080/oauth/authorize
Ein Zugangs-Token erhalten	http://localhost:8080/oauth/token

Profil

Aussehen

Sprache

Avatar

Passwort & Authentifizierung

Benachrichtigungen

Abwesenheit

Übersichten

Kalender

Geräte

Token-Zugriff

Verknüpfte Konten

Token-Zugriff

Generieren Sie ein persönliches Zugangs-Token für jede Anwendung, welche auf die Zammad API zugreifen soll.

Hinzufügen eines persönlichen Zugangs-Tokens

Wählen Sie den Namen der Applikation, danach wird ein eindeutiger Token generiert.

Erstellen

Persönliche Zugangs-Token

NAME	BERECHTIGUNGEN	ERSTELLT	LÄUFT AB	ZULETZT VERWENDET	LÖSCHEN
Script	admin, report, user_preferences	vor 2 Tagen 5 Stunden	in 203 Tagen	vor 1 Tag 5 Stunden	
Token	admin, report, user_preferences	vor 2 Tagen 4 Stunden	in 209 Tagen	vor 2 Tagen 4 Stunden	
IHK-API-Integration	admin, report, user_preferences	vor 2 Tagen 5 Stunden	in 202 Tagen		

```

Ticket_erstellensh X  docker-compose.yml  .env
scripts > $ Ticket_erstellensh > api_info
1  #!/bin/bash
2  source "${dirname "$0"}/.env"
3
4  echo "ZAMMAD API TICKET ERSTELLEN"
5  echo "=====
6
7  read -p "Titel des Tickets: " TITEL
8  read -p "Kunde/Name: " KUNDE
9  read -p "Seit wann besteht das Problem?: " SEIT_WANN
10 read -p "Betroffenes Gerät oder Anwendung: " GERAET
11 read -p "Fehlermeldung oder Beobachtung: " FEHLER
12 read -p "Auswirkung auf die Arbeit: " AUSWIRKUNG
13 read -p "Was wurde bereits versucht?: " VERSUCHT
14
15 api_info="Ticket durch API erstellt!"
16
17 BESCHREIBUNG=" $api_info | Benutzer: $KUNDE | Das Problem besteht seit: $SEIT_WANN | Gerät/Anwendung: $GERAET | Fehlermeldung/Beobachtung: $FEHLER | Auswirkung: $AUSWIRKUNG"
18
19 curl -X POST "$ZAMMAD_URL/api/v1/tickets" \
20 -u "$ZAMMAD_USER:$ZAMMAD_PASSWORD" \
21 -H "Content-Type: application/json" \
22 -d "{
23   \"title\": \"$TITEL\",
24   \"group_id\": 2,
25   \"customer_id\": 3,
26   \"article\": {
27     \"subject\": \"$TITEL\",
28     \"body\": \"$BESCHREIBUNG\",
29     \"type\": \"note\",
30     \"internal\": false
31   },
32   \"priority_id\": 2,
33   \"state_id\": 1
34 }"

```

The screenshot shows the Zammad web interface. At the top, there's a user profile and a settings icon. The main heading is "Automatisch erstelltes Ticket: Monitoring Warnung" with a subtext "Ticket#40046 - erstellt vor 1 Tag 5 Stunden". Below this, there's a message from AI: "Dieses Ticket wurde automatisch über die API erstellt." with buttons "auf intern setzen" and "abspalten". A status bar indicates "Priorität ist Kritisch" with buttons "auf öffentlich setzen" and "abspalten". At the bottom, there's a section for "Notiz schreiben, oder Dateien wählen..." with a user profile icon.

23. Backup- und Restore-Konzept

In Zammad sind folgende Daten besonders wichtig:

- Tickets
- Benutzer
- Organisationen
- Gruppen
- Rollen
- interne Notizen

- Kundenantworten
- Knowledge-Base-Artikel
- Trigger und Automatisierungen
- Systemeinstellungen

Ein Datenverlust hätte folgende Folgen:

- offene Tickets wären verloren
- Supportverlauf wäre nicht mehr nachvollziehbar
- Knowledge-Base-Inhalte wären nicht verfügbar
- Benutzer und Rechte müssten neu erstellt werden
- Nachweise für die Projektarbeit könnten verloren gehen

Backup-Prüfung:

`docker volume ls`

`docker-compose ps`

`docker-compose logs zammad-backup --tail=100`

Backups sollten regelmäßig erstellt werden, zum Beispiel täglich. Die Kontrolle der Backups sollte durch die Systemadministration erfolgen.

Ein Restore sollte geplant und kontrolliert durchgeführt werden. Vor einem Restore sollte geprüft werden:

- Welcher Datenstand wird benötigt?
- Welche Volumes sind betroffen?
- Gibt es aktuelle Backups?
- Wird das produktive System vorher gestoppt?
- Wurde der Restore getestet?

Risiko bei falschem Restore:

Ein falscher Restore kann aktuelle Tickets überschreiben oder Daten auf einen alten Stand zurücksetzen. Deshalb muss ein Restore immer dokumentiert und vorher geprüft werden.


```

hacimah@TABLET-I2N3ACEC:~/Abschlussprojekt_IT-Support/docker$ docker volume ls
DRIVER      VOLUME NAME
local       68a5876d324d793a639707a043d10e4b978744bda20ce8b6df7146a52a490c7e
local       docker_elasticsearch-data
local       docker_postgresql-data
local       docker_redis-data
local       docker_zammad-backup
local       docker_zammad-storage
hacimah@TABLET-I2N3ACEC:~/Abschlussprojekt_IT-Support/docker$ docker-compose ps
          Name                                Command                                State                                Ports
-----
docker_zammad-backup_1                       /opt/zammad/bin/docker-ent ...      Up
docker_zammad-elasticsearch_1                /bin/tini -- /usr/local/bi ...      Up                                9200/tcp, 9300/tcp
docker_zammad-init_1                         /opt/zammad/bin/docker-ent ...      Exit 0
docker_zammad-memcached_1                    docker-entrypoint.sh memca ...      Up (healthy)                    11211/tcp
docker_zammad-nginx_1                        /opt/zammad/bin/docker-ent ...      Up                                0.0.0.0:8080->8080/tcp,:::8080->8080/tcp
docker_zammad-postgresql_1                   docker-entrypoint.sh postgres ...    Up (healthy)                    5432/tcp
docker_zammad-railsserver_1                  /opt/zammad/bin/docker-ent ...      Up (healthy)
docker_zammad-redis_1                        docker-entrypoint.sh redis ...      Up (healthy)                    6379/tcp
docker_zammad-scheduler_1                    /opt/zammad/bin/docker-ent ...      Up
docker_zammad-websocket_1                    /opt/zammad/bin/docker-ent ...      Up
hacimah@TABLET-I2N3ACEC:~/Abschlussprojekt_IT-Support/docker$ docker-compose logs zammad-backup | tail -100
Attaching to docker_zammad-backup_1
zammad-backup_1 | Checking if PostgreSQL is ready...
zammad-backup_1 | Checking if Zammad is ready...
zammad-backup_1 | Starting backup loop..
zammad-backup_1 | 20260611114403 - backing up zammad...
zammad-backup_1 | tar: Removing leading `/' from member names
zammad-backup_1 | backup finished :)
zammad-backup_1 | sleeping 54933 seconds until the next backup run...
hacimah@TABLET-I2N3ACEC:~/Abschlussprojekt_IT-Support/docker$ █

```

24. Testprotokoll

Nr. Testfall	Erwartetes Ergebnis	Ergebnis
1 Zammad startet	Loginseite erreichbar	erfolgreich
2 Admin Login	Admin kann sich anmelden	erfolgreich
3 Docker Neustart	Daten bleiben erhalten	erfolgreich
4 Ticket manuell erstellen	Ticket wird gespeichert	erfolgreich
5 Ticket schließen	Status wird geändert	erfolgreich
6 Gruppe zuweisen	Ticket landet in richtiger Gruppe	erfolgreich
7 Priorität ändern	Priorität wird gespeichert	erfolgreich
8 Passwort Trigger	Benutzerverwaltung wird gesetzt	erfolgreich
9 VPN Trigger	Netzwerk Support wird gesetzt	erfolgreich
10 Drucker Trigger	Hardware Support wird gesetzt	erfolgreich
11 Kritisch Trigger	Systemadministration wird informiert	erfolgreich
12 Automatisierung alte Tickets	Tag wird gesetzt	erfolgreich
13 Knowledge Base Artikel öffnen	Artikel ist sichtbar	erfolgreich

Nr. Testfall	Erwartetes Ergebnis	Ergebnis
14 Knowledge Base im Ticket verwenden	Kunde wird auf Artikel verwiesen	erfolgreich
15 API Ticket erstellen	Ticket wird über Skript erstellt	erfolgreich
16 API Fehler falscher Token	Fehler wird erkannt	erfolgreich
17 API Fehler falsches Passwort	Fehler wird erkannt	erfolgreich
18 Backup Prüfung	Backup Container vorhanden	erfolgreich
19 Git Log prüfen	sinnvolle Commits vorhanden	erfolgreich
20 Eskalation testen	Ticket wird weitergeleitet	erfolgreich
21 Kundenantwort prüfen	Antwort ist verständlich	erfolgreich
22 Warten auf Kunde	Status wird gesetzt	erfolgreich
23 Kritisches Ticket	Priorität kritisch gesetzt	erfolgreich
24 Trigger mit API-Ticket prüfen	Zuordnung wird geprüft	erfolgreich
25 Browsercache-Problem lösen	Seite wieder erreichbar	erfolgreich

25. Fehlerdokumentation

Fehler 1: docker compose nicht verfügbar

Problem:

docker compose version

führte zu:

docker: unknown command: docker compose

Analyse:

Docker Compose war nicht als Docker-Plugin installiert, sondern als eigener Befehl verfügbar.

Lösung:

docker-compose --version

Danach wurde im Projekt docker-compose verwendet.

Fehler 2: Zammad zunächst nicht über Browser erreichbar

Problem:

Zammad war zeitweise im Browser nicht erreichbar.

Analyse:

Die Container liefen und curl zeigte:

HTTP/1.1 200 OK

Damit war klar, dass Zammad serverseitig erreichbar war.

Lösung:

Der Browsercache wurde gelöscht. Danach war Zammad wieder erreichbar.

Erkenntnis:

Nicht jeder Verbindungsfehler liegt am Server. Auch lokale Browserdaten können Probleme verursachen.

Fehler 3: localhost ohne Port zeigt Apache

Problem:

Beim Aufruf von:

http://localhost

wurde eine Apache-Seite angezeigt.

Analyse:

Port 80 wurde durch Apache verwendet. Zammad lief auf Port 8080.

Lösung:

Zammad wurde über folgende Adresse geöffnet:

http://localhost:8080

Fehler 4: CSRF-Token fehlgeschlagen

Problem:

Während der Admin-Erstellung erschien:

Die Verifizierung des CSRF-Tokens ist fehlgeschlagen.

Lösung:

Seite neu geladen und Ersteinrichtung erneut durchgeführt.

Fehler 5: API Token authorization failed

Problem:

Ein API-Token funktionierte bei GET-Abfragen, aber nicht zuverlässig bei POST-Anfragen zur Ticketerstellung.

Analyse:

GET /api/v1/users/me war erfolgreich. POST /api/v1/tickets schlug fehl.

Lösung:

Die API-Integration wurde über HTTP Basic Authentication umgesetzt.

Fehler 6: Invalid BasicAuth credentials

Problem:

Bei der Basic-Auth-Variante erschien:

Invalid BasicAuth credentials

Analyse:

Das Passwort enthielt Sonderzeichen und war in der .env nicht korrekt verarbeitet.

Lösung:

Die Werte in der .env wurden in Anführungszeichen gesetzt.

Fehler 7: curl URL rejected

Problem:

curl zeigte:

Malformed input to a URL function

Analyse:

Die .env-Datei enthielt Windows-Zeilenenden ^M.

Nachweis:

cat -A .env

Lösung:

sed -i 's/\r\$//' .env

sed -i 's/\r\$//' script.sh

Fehler 8: Gruppe nicht gefunden

Problem:

Bei der API-Ticketerstellung erschien:

No lookup value found for 'group': "Systemadministration"

Analyse:

Die Gruppe hieß in Zammad nicht nur Systemadministration, sondern war als Untergruppe angelegt.

Lösung:

Statt Gruppenname wurde die Gruppen-ID verwendet:

"group_id": 8

26. Bewertung der Lösung

Die eingeführte Lösung verbessert den IT-Support der OfficeLine GmbH deutlich.

Vorher:

- Anfragen kamen unstrukturiert an
- Zuständigkeiten waren unklar
- keine zentrale Dokumentation
- wiederkehrende Probleme wurden mehrfach erklärt

Nachher:

- alle Anfragen werden als Tickets erfasst
 - Gruppen und Rollen regeln Zuständigkeiten
 - Trigger unterstützen automatische Zuordnung
 - Automatisierungen verhindern Liegezeiten
 - Knowledge Base ermöglicht Selbsthilfe
 - API ermöglicht automatische Ticketerstellung
 - GitHub dokumentiert Projektfortschritt
 - Backup-Konzept reduziert Risiko von Datenverlust
-

27. Live-Demo Ablauf

Für die Präsentation ist folgende Live-Demo geplant:

1. Zammad öffnen
2. Ticketübersicht zeigen
3. Beispiel-Ticket öffnen
4. automatische Zuordnung durch Trigger erklären
5. Knowledge-Base-Artikel öffnen
6. API-Skript im Terminal ausführen
7. automatisch erstelltes Ticket in Zammad zeigen

8. kurz Backup/Testnachweise erwähnen

Die Live-Demo zeigt nicht nur die Startseite, sondern einen vollständigen Ablauf im Ticketsystem.

28. Fazit

Im Projekt wurde ein vollständiges internes Ticketsystem mit Zammad aufgebaut. Die technische Grundlage wurde mit Docker Compose umgesetzt. Durch Gruppen, Rollen, Tickets, Trigger und Automatisierungen wurde ein realistischer IT-Support-Prozess abgebildet.

Die Knowledge Base unterstützt Benutzer bei Standardproblemen und entlastet den Support. Die API-Integration zeigt, wie externe Systeme automatisch Tickets erstellen können. Durch Git, Screenshots, Testprotokoll und Fehlerdokumentation ist das Projekt nachvollziehbar dokumentiert.

Besonders wichtig war nicht nur die technische Umsetzung, sondern auch das Verständnis der Supportprozesse. Fehler wurden nicht ignoriert, sondern analysiert und dokumentiert. Dadurch ist das Projekt praxisnah und nachvollziehbar.

THE END